Setup and Compatibility:

The program was run on WSL (Ubuntu) version 2, on a computer with an AMD Ryzen 5 5500, 3600 Mhz, 6 Core(s), 12 Logical Processor(s) CPU, and 32 GB DDR4 RAM. g++ 13.3.0 was used to compile the program with the command “g++ -O2 main.cpp valuation.cpp -o main” on the non-parallel implementations (iter4 and before) and “g++ -O2 -fopenomp main.cpp valuation.cpp -o main” for the parallel implementation (iter5).

Initial Parameters and Benchmarks:

To properly compare my implementation with the results provided by the paper, the initial parameters used were the conditions given by Chapter 6 of the paper, where lower and upper barriers L = 90 and U = 110, strike price K = 100, maturity T = 0.5, spot price S_0 = 100, risk-neutral drift = 0.1, and volatility σ = 0.3.

Additionally, each stimulation uses 100 000 sample instances and is repeated 50 times to derive the standard error. These stimulations will be conducted with a varying number of “steps” where the samples are propagated, and resampled steps, with the number of steps being N, where N ∈ {1, 2, 4, 8, 16, 32, 64, 128}.

The relative efficiency of the standard Monte Carlo (SMC) vs standard Monte Carlo (MC) estimators will be given by the coefficient κ, where:

$$\kappa = \frac{s_{MC}^{2} t_{MC}}{S_{SMC}^{2} t_{SMC}},$$ where *s* is the standard error, *t* is the time taken to complete a simulation

Furthermore, the paper uses ψ to represent the probability of a sampled asset hitting the barrier.

1. Discretely Monitored Barrier:

Results from Paper:

N	MC(stderr)	SMC(stderr)	κ	ψ
1	0.8225(0.11%)	0.8229(0.12%)	0.69	0.359
2	0.5146(0.16%)	0.5140(0.10%)	2.11	0.229
4	0.2985(0.16%)	0.2985(0.10%)	2.19	0.137
8	0.1675(0.27%)	0.1684(0.11%)	4.98	0.080
16	0.0952(0.33%)	0.0957(0.11%)	7.25	0.048
32	0.0568(0.44%)	0.0566(0.13%)	10.54	0.029
64	0.0358(0.57%)	0.0361(0.13%)	17.84	0.019
128	0.0246(0.66%)	0.0249(0.14%)	20.12	0.013

My Results:

N	SMC Price	StdErr (%)	MC Price	StdErr (%)	Survival Rate	k
1	0.8245	0.11	0.8218	0.129	0.360	0.94
2	0.5138	0.12	0.5139	0.145	0.230	1.10
4	0.2984	0.11	0.2982	0.218	0.137	1.95
8	0.1681	0.10	0.1686	0.270	0.081	2.67
16	0.0957	0.10	0.0956	0.340	0.048	2.83
32	0.0568	0.14	0.0569	0.411	0.030	4.43
64	0.0361	0.10	0.0359	0.544	0.020	7.78
128	0.0249	0.12	0.0248	0.696	0.014	8.36

While most values, such as the price, stderr, and survival rate of my implementation matches the paper's results, my SMC implementation has a substantially lower relative efficiency compared to the paper's SMC relative efficiency.

2. Continuously Monitored Barrier:

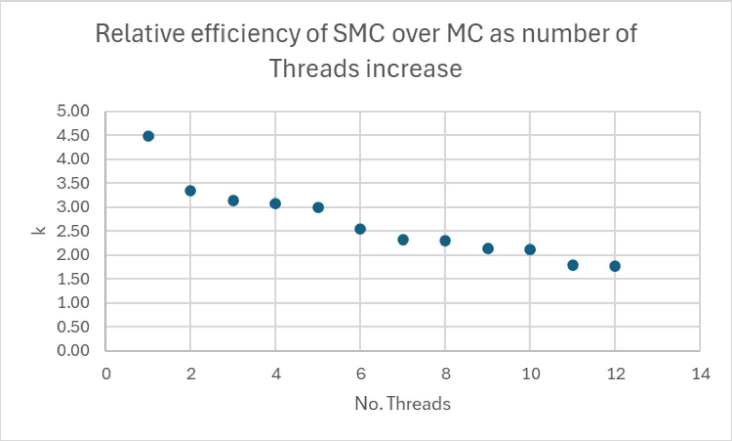
N	MC(stderr)	SMC(stderr)	κ
1	0.008069(0.10%)	0.008074(0.12%)	0.39
2	0.008059(0.19%)	0.008077(0.13%)	1.14
4	0.008059(0.29%)	0.008064(0.14%)	1.58
8	0.008033(0.43%)	0.008046(0.15%)	3.98
16	0.008027(0.58%)	0.008066(0.12%)	8.15
32	0.008098(0.67%)	0.008063(0.13%)	13.25
64	0.008001(0.77%)	0.008070(0.13%)	16.12
128	0.007953(1.01%)	0.008050 (0.14%)	23.84

N	SMC Price	StdErr (%)	MC Price	StdErr (%)	k
1	0.008059	0.11	0.008060	0.10	0.56
2	0.008085	0.12	0.008059	0.16	0.97
4	0.008059	0.12	0.008101	0.28	1.56
8	0.008063	0.15	0.008083	0.40	1.72
16	0.008060	0.19	0.008043	0.53	2.02
32	0.008061	0.18	0.008076	0.76	2.61
64	0.008067	0.16	0.007998	0.83	2.81
128	0.008076	0.14	0.007983	0.85	4.48

Here price, standard error, and survival rate between the two implementations are the same, while the coefficient κ shows that my implementation of SMC, which is around 3x more efficient than MC at N = 128, is even less efficient compared to the paper's implementation, whose SMC is around 24x more efficient than the MC. I have no idea why this is the case, but my SMC still improves in relative efficiency as the number of steps increases.

3. Continuously Monitored Barrier, Parallel Computation:

N	SMC Price	StdErr (%)	MC Price	StdErr (%)	k
1	0.008061	0.13	0.00806089	0.11	0.19
2	0.008071	0.12	0.00805188	0.12	0.23
4	0.008058	0.13	0.00808307	0.33	0.79
8	0.008072	0.14	0.00806765	0.42	0.95
16	0.008070	0.18	0.0080242	0.54	0.98
32	0.008071	0.14	0.00801931	0.88	1.27
64	0.008063	0.17	0.00810636	0.90	1.42
128	0.008062	0.15	0.00818112	0.85	1.76



Applying parallel processing with 12 threads to the continuously monitored barrier models leads to a decrease in run time by around 7 times when applied to the standard Monte Carlo algorithm, and decreases the run time of the sequential Monte Carlo by 4 times. Consequently, the gap in efficiency between SMC and MC closes further when processes are parallelised, with SMC being only 1.76x more efficient than MC at N = 128. This is because individual samples in standard Monte Carlo are independent from each other, which allows them to be easily run in parallel. On the other hand, for SMC, while the propagation and resampling step can both be run in parallel, the setup to partition the arrays for parallel resampling, such as the generation of a prefix sum array, must be done sequentially. Additionally, the threads must be synchronised throughout different points in the resampling step to avoid race conditions. These steps contribute to increased overhead costs for running SMC in parallel. As seen in the graph above, as the number of threads increases, MC becomes more efficient relative to SMC as it is better able to utilise the multiple processors present.

Conclusion:

While the efficiency of SMC vs MC derived from my implementation was lower compared to the paper's implementation, the following observations made were consistent with the observations made in the paper:

- The standard error of the SMC model does not grow as the number of time steps increases, while the standard error for MC grows significantly.
- Relative SMC vs MC efficiency improves when the probability of explicitly crossing the barrier grows, as the resampling from SMC can prevent the waste of processing time from a sample path crossing the barrier being terminated without directly contributing to the final value.

Additionally:

- Due to the independent nature of MC, MC is better utilising multiple logical processors than SMC; hence, when the number of logical processors increases, the relative efficiency of SMC against MC decreases.